

Université Sorbonne Paris Nord
Ecole d'Ingénieurs Sup Galilée

COMPTE RENDU

TP4: Introduction au Langage Procédural (PL/SQL) d'Oracle

Réalisé par:

ELGARANI Hasna

2024/2025

Exercice n°1 :

1) Calcul de somme

```
DECLARE
a NUMBER;
b NUMBER;
BEGIN
a := TO_NUMBER('&Entrez_le_premier_nombre');
b := TO_NUMBER('&Entrez_le_deuxieme_nombre');
DBMS_OUTPUT.PUT_LINE('La somme est : ' || (a + b));
END;
/
```

2) Générateur de table de multiplication

```
DECLARE
n NUMBER;
i NUMBER := 1;
BEGIN
n := TO_NUMBER('&Entrez_un_nombre');
WHILE i <= 10 LOOP
DBMS_OUTPUT.PUT_LINE(n || ' x ' || i || ' = ' || (n * i));
i := i + 1;
END LOOP;
END;
/
```

3) Implémentation récursive d'une fonction mathématique

```
CREATE OR REPLACE FUNCTION puissance(x IN NUMBER, n IN NUMBER) RETURN NUMBER IS
BEGIN
IF n = 0 THEN
```

```

RETURN 1;

ELSE

RETURN x * puissance(x, n - 1);

END IF;

END;

/

```

4) Calcul et persistance de données

```

CREATE TABLE resultatFactoriel (nombre NUMBER, factoriel NUMBER);

DECLARE

n NUMBER;

fact NUMBER := 1;

i NUMBER;

BEGIN

n := TO_NUMBER('&Entrez_un_nombre_strictement_positif');

IF n <= 0 THEN

DBMS_OUTPUT.PUT_LINE('Le nombre doit être strictement positif');

ELSE

FOR i IN 1..n LOOP

fact := fact * i;

END LOOP;

INSERT INTO resultatFactoriel (nombre, factoriel) VALUES (n, fact);

DBMS_OUTPUT.PUT_LINE('La factorielle de ' || n || ' est : ' || fact);

END IF;

END;

/

```

5) Traitement par lots et stockage de résultats

```

CREATE TABLE resultatsFactoriels (nombre NUMBER, factoriel NUMBER);

DECLARE

```

```

fact NUMBER := 1;

BEGIN

FOR i IN 1..20 LOOP

fact := 1;

FOR j IN 1..i LOOP

fact := fact * j;

END LOOP;

INSERT INTO resultatsFactoriels (nombre, factoriel) VALUES (i, fact);

END LOOP;

DBMS_OUTPUT.PUT_LINE('Les factorielles ont été calculées et stockées.');
```

END;

/

Exercice n°2 :

1)Ajout d'un nouvel employé

```

DECLARE

v_employe emp%ROWTYPE;

BEGIN

v_employe.matr := 4;

v_employe.nom := 'Youcef';

v_employe.sal := 2500;

v_employe.adresse := 'avenue de la République';

v_employe.dep := 92002;

INSERT INTO emp VALUES v_employe;

DBMS_OUTPUT.PUT_LINE('Employé inséré avec succès.');
```

END;

/

2) Suppression des employés d'un département donné

```
DECLARE

v_nb_lignes NUMBER;

BEGIN

DELETE FROM emp WHERE dep = 10;

v_nb_lignes := SQL%ROWCOUNT;

DBMS_OUTPUT.PUT_LINE('Nombre d"employés supprimés : ' || v_nb_lignes);

END;

/
```

3) Calcul de la somme des salaires avec curseur et boucle

```
SET SERVEROUTPUT ON;

DECLARE

v_salaire emp.sal%TYPE;

v_total emp.sal%TYPE := 0;

CURSOR c_salaires IS SELECT sal FROM emp;

BEGIN

OPEN c_salaires;

LOOP

FETCH c_salaires INTO v_salaire;

EXIT WHEN c_salaires%NOTFOUND;

IF v_salaire IS NOT NULL THEN

v_total := v_total + v_salaire;

END IF;

END LOOP;

CLOSE c_salaires;

DBMS_OUTPUT.PUT_LINE('Somme des salaires : ' || v_total);

END;

/
```

4) Calcul du salaire moyen avec boucle LOOP

```
SET SERVEROUTPUT ON;

DECLARE

  v_salaire emp.sal%TYPE;

  v_total emp.sal%TYPE := 0;

  v_count NUMBER := 0;

  CURSOR c_salaires IS SELECT sal FROM emp;

BEGIN

  OPEN c_salaires;

  LOOP

    FETCH c_salaires INTO v_salaire;

    EXIT WHEN c_salaires%NOTFOUND;

    IF v_salaire IS NOT NULL THEN

      v_total := v_total + v_salaire;

      v_count := v_count + 1;

    END IF;

  END LOOP;

  CLOSE c_salaires;

  IF v_count > 0 THEN

    DBMS_OUTPUT.PUT_LINE('Salaire moyen : ' || (v_total / v_count));

  ELSE

    DBMS_OUTPUT.PUT_LINE('Aucun salaire trouvé.');
```

END IF;

END;

/

5) utilisant la boucle FOR...IN

→ Calcul de la somme des salaires

```

SET SERVEROUTPUT ON;

DECLARE

v_total emp.sal%TYPE := 0;

CURSOR c_salaires IS SELECT sal FROM emp;

BEGIN

FOR ligne IN c_salaires LOOP

IF ligne.sal IS NOT NULL THEN

v_total := v_total + ligne.sal;

END IF;

END LOOP;

DBMS_OUTPUT.PUT_LINE('Somme des salaires (FOR) : ' || v_total);

END;

/

```

→Calcul de la moyenne des salaires

```

SET SERVEROUTPUT ON;

DECLARE

v_total emp.sal%TYPE := 0;

v_count NUMBER := 0;

v_moyenne NUMBER;

CURSOR c_salaires IS SELECT sal FROM emp;

BEGIN

FOR ligne IN c_salaires LOOP

IF ligne.sal IS NOT NULL THEN

v_total := v_total + ligne.sal;

v_count := v_count + 1;


```

```

END IF;

END LOOP;

IF v_count > 0 THEN

    v_moyenne := v_total / v_count;

    DBMS_OUTPUT.PUT_LINE('Moyenne des salaires (FOR) : ' || v_moyenne);

ELSE

    DBMS_OUTPUT.PUT_LINE('Aucun salaire à traiter.');
```

END IF;

END;

/

6) Procédure anonyme avec curseur paramétré

```

SET SERVEROUTPUT ON;

DECLARE

    CURSOR c(p_dep emp.dep%TYPE) IS

        SELECT dep, nom FROM emp WHERE dep = p_dep;

BEGIN

    FOR rec IN c(10) LOOP

        DBMS_OUTPUT.PUT_LINE('Département : ' || rec.dep || ', Employé : ' || rec.nom);

    END LOOP;

END;
```

/

Exercice n°3 :

→ Création d'un package PL/SQL pour la gestion des clients

Dans ce troisième exercice, nous avons conçu un package qui permet de gérer les clients dans une base de données Oracle, avec une utilisation de la surcharge de procédures.

Spécification du package

```
CREATE OR REPLACE PACKAGE gestion_clients AS
```

```
    PROCEDURE ajouter_client(p_id_client NUMBER, p_nom VARCHAR2, p_adresse VARCHAR2);
```

```
    PROCEDURE ajouter_client(p_nom VARCHAR2, p_adresse VARCHAR2);
```

```
END gestion_clients;
```

```
/
```

Corps du package

```
CREATE OR REPLACE PACKAGE BODY gestion_clients AS
```

```
    PROCEDURE ajouter_client(p_id_client NUMBER, p_nom VARCHAR2, p_adresse VARCHAR2) IS
```

```
        v_count NUMBER;
```

```
    BEGIN
```

```
        -- Vérification de l'unicité de l'ID client
```

```
        SELECT COUNT(*) INTO v_count FROM clients WHERE id_client = p_id_client;
```

```
        IF v_count > 0 THEN
```

```
            RAISE_APPLICATION_ERROR(-20001, 'Cet ID client est déjà utilisé. Merci d'en choisir un autre.');
```

```
        END IF;
```

```
        -- Insertion du nouveau client
```

```
        INSERT INTO clients (id_client, nom, adresse)
```

```
        VALUES (p_id_client, p_nom, p_adresse);
```

```
        -- Confirmation et validation
```

```
        DBMS_OUTPUT.PUT_LINE('Ajout du client réussi avec l'ID : ' || p_id_client);
```

```
        COMMIT;
```

```
    EXCEPTION
```

```
        WHEN OTHERS THEN
```

```
            DBMS_OUTPUT.PUT_LINE('Erreur lors de l'ajout du client : ' || SQLERRM);
```

```
    END;
```

```
PROCEDURE ajouter_client(p_nom VARCHAR2, p_adresse VARCHAR2) IS  
  
    v_id_client NUMBER;  
  
BEGIN  
  
    -- Génération d'un nouvel ID client (logique à définir selon la politique de l'entreprise)  
  
    -- Insertion du client avec génération automatique de l'ID  
  
    NULL; -- À compléter selon le besoin  
  
END;  
  
END gestion_clients;  
  
/
```